

LLM Password Router: Final Report

ECE 202C project - password semantic vulnerability checked with LLM token probabilities

Problem

Widely used password meters are strong at explicit surface rules, but they can miss passwords that are easy because of meaning. A password can be long, mixed-looking, and still generated by a short human rule: code fragments, familiar number sequences, address-like strings, fandom terms, or semantic word chains.

Threat model

- Attacker has an offline password hash leak: username/email, salt, and stored password hash.
- For each guess, the attacker computes $\text{hash}(\text{guess}, \text{salt})$ and compares it locally.
- Salt prevents one reusable hash table, but not an ordered guess generator. The guess order is the real target.
- A strong semantic attacker can be seen as lowest-cost traversal over an implicit tree of constructions: append likely chunks, give each edge cost $-\log_{10} P(\text{edge} \mid \text{prefix})$, and try complete strings in increasing cost.

```
node: partial password string / model context
edge: append likely token, word, digit block, symbol, or mutation
edge cost:  $-\log_{10} P(\text{edge} \mid \text{prefix})$ 
defense question: would this password appear early in that guess order?
```

Research question

Can a small LLM be used as a probability model for semantic guessability, without asking it to make a direct subjective judgement?

Approach

Router = cheap prechecks first, LLM token-DP only for passwords that survive the cheap floor

Pipeline

```
raw password
-> Stage 0: zxcvbn + stable substrings + bad substring resource + pwned variants
-> Qwen2.5-1.5B token-DP on two continuation templates
-> optional Mistral-7B token-DP if the 1.5B result is borderline
-> reject / accept / autoaccept
```

LLM score

The LLM stage estimates a prefix cost: $\text{score} = -\log_{10} P(\text{model output starts with the password})$. Lower scores are more suspicious, because the model assigns more probability to generating the exact string. The final demo scores one raw target string; it does not ask the user for separators or scan every possible split.

Why token dynamic programming

One visible string can be generated through multiple token paths. For example, cat9 might be generated as [cat][9] or [c][at][9]. The DP sums the probability mass of matching token paths instead of trusting only one tokenization. The implementation keeps a bounded number of token paths per character index, so the method is practical but still explicitly approximate.

```
cat9:
[cat][9]      cost 1.2 + 0.7 = 1.9
[c][at][9]    cost 0.8 + 1.6 + 0.7 = 3.1
 $P = 10^{-1.9} + 10^{-3.1}$ , score  $\approx 1.87$ 
```

Prompting decision

The useful prompts put the model in a continuation setting, such as "Very long but vulnerable password: <prefix>" or a task field with "Prefix:" and "Continuation:". Direct judgement prompts were tested separately and rejected as a deployment method.

Experiments and Results

Controlled experiments, negative results, and final interactive demo behavior

Experiment 1D: token-DP signal

The bundled Experiment 1D result set uses 100 synthetic passwords and two models. It is a controlled continuation experiment: an annotated known prefix is given and the model scores the remainder. That makes it useful for studying semantic continuation, while the final router demo separately scores whole raw passwords from an empty automatic prefix.

Condition	n	avg zxcvbn	avg best LLM
linked	45	9.86	6.50
control	45	9.47	7.45
random	10	9.78	8.45

Experiment 2: prompting matters

Continuation templates gave a more usable signal than directly asking whether a password was structured. The direct yes/no structural variants were prompt-sensitive and had weak separation, so they were not used in the final router.

Experiment 2B: direct JSON ratings failed

A tempting shortcut is to ask a small LLM to return a JSON risk_score. This failed badly enough to keep as a negative result. The best direct-rating approach reached AUC about 0.81, but false positives remained high. Random-looking strings received high risk scores such as 85 or 95 under the direct-rating template.

Example direct-rating failure	Score
HNY6TZkoqskfD0e!	85
GxZ!@Mj&_8XXs*	95
-Wup!6Oa4SZR8oN2BhkQ7K	95
-Aweb%!1XK#G#!	95

Takeaway: do not ask the LLM for an opinion. Measure what it would generate.

Final Demo Behavior

Original router and capped variant are both included in the repository

Original demo thresholds

Stage 0 rejects if zxcvbn or patched Stage 0 log10 is below 10. The uncapped LLM-DP rejects below 20 and autoaccepts above 32. Borderline passwords escalate from Qwen 1.5B to Mistral 7B.

Class	Example	Score / outcome
Code fragment	for (int i = 0; i < n; i++)	14.57 reject
Math-looking	727sqrt(621.136)	20.63 then later accept
Random-looking	yOn7s78QJ?<c	32.37 autoaccept
Stage 0 semantic chunks	wolfgoarfarf	patched Stage 0 reject

Capped variant

The capped notebook adds fallback edges with cost $C * \text{extra_chars}$, with $C = 2.5$. This prevents weird random suffixes from receiving unlimited LLM surprise credit, while still letting a sufficiently random prefix or suffix rescue a semantic core. The capped policy rejects below 15 and autoaccepts above 25.

```
score[i] <= min_{0 <= j < i} score[j] + 2.5 * (i - j)
interpretation: semantic prefix plus brute-force-ish suffix fallback
status: experimental policy variant, kept alongside the original demo
```

Example capped progression

Password pattern	Capped score	Verdict
3-6-9-12-15-18-21	6.40	reject
3-6-9-12-15-18-21m7*	13.30	reject
3-6-9-12-15-18-21m7*>	20.09 final	accept
2*j^gayfurrywolf	27.04	autoaccept

Limitations, AI Limitations, and Conclusion

What this project does not prove, and what should happen next

Limitations

- The LLM-DP score is model- and prompt-specific. It is not a literal crack-time estimate.
- The bounded token-path DP is approximate. Pruned paths were checked in experiments, but the method should still be described as bounded beam/path-sum DP rather than exact exhaustive probability.
- Private or subculture-specific meanings can remain invisible unless the relevant dictionary/pattern is added to Stage 0 or the LLM already knows them.
- The main datasets are synthetic or controlled. A next evaluation should use a larger public fake-password benchmark and a broader set of generated semantic patterns.

Limitations of AI in the project

LLMs were useful as probability models, but unreliable as direct judges. The direct JSON-rating experiment showed that a small model can confidently assign high risk to random-looking strings. AI was also useful for development iteration, but all generated code needed verification because small implementation changes changed the meaning of the DP state.

Conclusion

The project produced a password-checking router that combines cheap rule-based prechecks with LLM token-probability scoring. The most important result is not that LLMs can magically grade password strength. It is narrower: continuation probability can expose semantic predictability that finite pattern matchers miss, while direct LLM judgement is not reliable enough to use as the security decision. The repository includes the original demo, the capped variant, experiment notebooks, result files, and AI-interaction evidence embedded in the method and evaluation.

Repository: <https://github.com/Aarreks/llm-password/>